main Last_Believe / ADVArray.js

iamSaikrishna143 all update ce579bf · 9 hours ago

2201 lines (1890 loc) · 56.8 KB

Code Blame Raw

```
// Sure! I can provide JavaScript implementations for all 25 string problems without usi

// 1. Reverse a String
function reverseString(str) {
    let reversed = '';
    for (let i = str.length - 1; i >= 0; i--) {
        reversed += str[i];
    }
    return reversed;
}

// 2. Check if a String is a Palindrome
function isPalindrome(str) {
    for (let i = 0; i < str.length / 2; i++) {
        if (str[i] !== str[str.length - 1 - i]) return false;
    }
    return true;
}

// 3. Remove Duplicates from a String
function removeDuplicates(str) {
    let result = '';
    for (let i = 0; i < str.length; i++) {
        let found = false;
        for (let j = 0; j < result.length; j++) {
            if (str[i] === result[j]) {
                found = true;
                break;
            }
        }
        if (!found) result += str[i];
    }
    return result;
}

// 4. Find the First Non-Repeating Character
```

```
function firstNonRepeatingChar(str) {
    for (let i = 0; i < str.length; i++) {
        let unique = true;
        for (let j = 0; j < str.length; j++) {
            if (i !== j && str[i] === str[j]) {
                unique = false;
                break;
            }
        }
        if (unique) return str[i];
    }
    return null;
}

// 5. Count the Occurrences of Each Character
function charCount(str) {
    let count = {};
    for (let i = 0; i < str.length; i++) {
        let c = str[i];
        if (count[c] === undefined) count[c] = 1;
        else count[c]++;
    }
    return count;
}

// 6. Reverse Words in a Sentence
function reverseWords(sentence) {
    let word = '', result = '';
    for (let i = sentence.length - 1; i >= 0; i--) {
        if (sentence[i] === ' ') {
            result += word + ' ';
            word = '';
        } else {
            word = sentence[i] + word;
        }
    }
    result += word;
    return result;
}

// 7. Check if Two Strings are Anagrams
function areAnagrams(str1, str2) {
    if (str1.length !== str2.length) return false;
    let count1 = {}, count2 = {};
    for (let i = 0; i < str1.length; i++) {
        count1[str1[i]] = (count1[str1[i]] || 0) + 1;
        count2[str2[i]] = (count2[str2[i]] || 0) + 1;
    }
    for (let key in count1) {
        if (count1[key] !== count2[key]) return false;
    }
    return true;
```

```
}

// 8. Find the Longest Substring Without Repeating Characters
function longestUniqueSubstring(str) {
    let maxLength = 0, start = 0;
    for (let i = 0; i < str.length; i++) {
        for (let j = i; j < str.length; j++) {
            let unique = true;
            for (let k = i; k < j; k++) {
                if (str[k] === str[j]) {
                    unique = false;
                    break;
                }
            }
            if (unique && j - i + 1 > maxLength) {
                maxLength = j - i + 1;
                start = i;
            }
        }
    }
    let result = '';
    for (let i = start; i < start + maxLength; i++) result += str[i];
    return result;
}

// 9. Convert a String to an Integer (atoi Implementation)
function stringToInteger(str) {
    let num = 0, sign = 1, i = 0;
    if (str[0] === '-') { sign = -1; i++; }
    else if (str[0] === '+') i++;
    for (; i < str.length; i++) {
        let code = str[i].charCodeAt(0) - 48;
        if (code < 0 || code > 9) break;
        num = num * 10 + code;
    }
    return num * sign;
}

// 10. Compress a String (Run-Length Encoding)
function compressString(str) {
    let result = '', count = 1;
    for (let i = 0; i < str.length; i++) {
        if (str[i] === str[i+1]) count++;
        else {
            result += str[i] + count;
            count = 1;
        }
    }
    return result;
}

// 11. Find the Most Frequent Character
```

```
function mostFrequentChar(str) {
    let count = {}, maxChar = '', maxCount = 0;
    for (let i = 0; i < str.length; i++) {
        count[str[i]] = (count[str[i]] || 0) + 1;
        if (count[str[i]] > maxCount) {
            maxCount = count[str[i]];
            maxChar = str[i];
        }
    }
    return maxChar;
}

// 12. Find All Substrings of a Given String
function allSubstrings(str) {
    let result = [];
    for (let i = 0; i < str.length; i++) {
        for (let j = i; j < str.length; j++) {
            let sub = '';
            for (let k = i; k <= j; k++) sub += str[k];
            result.push(sub);
        }
    }
    return result;
}

// 13. Check if a String is a Rotation of Another String
function isRotation(str1, str2) {
    if (str1.length !== str2.length) return false;
    let combined = str1 + str1;
    for (let i = 0; i <= combined.length - str2.length; i++) {
        let match = true;
        for (let j = 0; j < str2.length; j++) {
            if (combined[i+j] !== str2[j]) { match = false; break; }
        }
        if (match) return true;
    }
    return false;
}

// 14. Remove All White Spaces from a String
function removeWhiteSpaces(str) {
    let result = '';
    for (let i = 0; i < str.length; i++) {
        if (str[i] !== ' ') result += str[i];
    }
    return result;
}

// 15. Check if a String is a Valid Shuffle of Two Strings
function isValidShuffle(str1, str2, shuffled) {
    let i=0, j=0;
    for (let k=0; k<shuffled.length; k++) {
```

```
            if (i<str1.length && shuffled[k] === str1[i]) i++;
            else if (j<str2.length && shuffled[k] === str2[j]) j++;
            else return false;
        }
        return i === str1.length && j === str2.length;
    }

    // 16. Convert a String to Title Case
    function toTitleCase(str) {
        let result = '', capitalize = true;
        for (let i = 0; i < str.length; i++) {
            if (str[i] === ' ') {
                result += ' ';
                capitalize = true;
            } else {
                result += capitalize ? str[i].toUpperCase() : str[i].toLowerCase();
                capitalize = false;
            }
        }
        return result;
    }

    // 17. Find the Longest Common Prefix
    function longestCommonPrefix(arr) {
        if (arr.length === 0) return '';
        let prefix = '';
        for (let i = 0; i < arr[0].length; i++) {
            let char = arr[0][i];
            for (let j = 1; j < arr.length; j++) {
                if (i >= arr[j].length || arr[j][i] !== char) return prefix;
            }
            prefix += char;
        }
        return prefix;
    }

    // 18. Convert a String to a Character Array
    function stringToCharArray(str) {
        let arr = [];
        for (let i = 0; i < str.length; i++) arr.push(str[i]);
        return arr;
    }

    // 19. Replace Spaces with %20 (URL Encoding)
    function urlEncode(str) {
        let result = '';
        for (let i = 0; i < str.length; i++) {
            result += (str[i] === ' ') ? '%20' : str[i];
        }
        return result;
    }

```

```
// 20. Convert a Sentence into an Acronym
function acronym(str) {
    let result = '', capitalize = true;
    for (let i = 0; i < str.length; i++) {
        if (str[i] === ' ') capitalize = true;
        else if (capitalize) { result += str[i].toUpperCase(); capitalize = false; }
    }
    return result;
}

// 21. Check if a String Contains Only Digits
function isOnlyDigits(str) {
    for (let i = 0; i < str.length; i++) {
        if (str[i] < '0' || str[i] > '9') return false;
    }
    return true;
}

// 22. Find the Number of Words in a String
function countWords(str) {
    let count = 0, inWord = false;
    for (let i = 0; i < str.length; i++) {
        if (str[i] !== ' ' && !inWord) { count++; inWord = true; }
        else if (str[i] === ' ') inWord = false;
    }
    return count;
}

// 23. Remove a Given Character from a String
function removeChar(str, charToRemove) {
    let result = '';
    for (let i = 0; i < str.length; i++) {
        if (str[i] !== charToRemove) result += str[i];
    }
    return result;
}

// 24. Find the Shortest Word in a String
function shortestWord(str) {
    let minLen = Infinity, len = 0;
    let word = '';
    for (let i = 0; i <= str.length; i++) {
        if (i < str.length && str[i] !== ' ') { word += str[i]; len++; }
        else {
            if (len > 0 && len < minLen) minLen = len;
            word = ''; len = 0;
        }
    }
    return minLen;
}

// 25. Find the Longest Palindromic Substring
```

```
297 function longestPalindromicSubstring(str) {
298     let maxLength = 0, start = 0;
299     for (let i = 0; i < str.length; i++) {
300         for (let j = i; j < str.length; j++) {
301             let palindrome = true;
302             for (let k = 0; k < (j - i + 1) / 2; k++) {
303                 if (str[i + k] !== str[j - k]) { palindrome = false; break; }
304             }
305             if (palindrome && j - i + 1 > maxLength) {
306                 maxLength = j - i + 1;
307                 start = i;
308             }
309         }
310     }
311     let result = '';
312     for (let i = start; i < start + maxLength; i++) result += str[i];
313     return result;
314 }
315
316
317 // ✅ All 25 solutions are implemented without using pre-built string manipulation func
318
319
320
321 // 1. Two Sum
322 // Given an array of integers, return indices of the two numbers such that they add up t
323 // You may assume that each input would have exactly one solution, and you may not use t
324 // You can return the answer in any order.
325 // Example:
326 // Input: nums = [2,7,11,15], target = 9
327 // Output: [0,1]
328 // Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
329 var twoSum = function (nums, target) {
330     for (let i = 0; i < nums.length; i++) {
331         for (let j = i+1; j < nums.length; j++) {
332             if (nums[i] + nums[j] === target) {
333                 return [i, j]
334             }
335
336         }
337
338     }
339     return nums
340 };
341 console.log(twoSum([2, 7, 11, 15], 9)); // Output: [0, 1]
342
343 // ----------------------------------------------------------------------------------
344
345 //2. Palindrome Number
346 // Given an integer x, return true if x is palindrome integer.
347 // An integer is a palindrome when it reads the same backward as forward.
348 // For example, 121 is a palindrome while 123 is not.
```

```
var isPalindrome = function (x) {
  if (x < 0) return false; // Negative numbers are not palindromes
  let reversed = 0;
  let original = x;
  while (original > 0) {
    reversed = reversed * 10 + (original % 10);
    original = Math.floor(original / 10);
  }
  return x === reversed;
};
console.log(isPalindrome(121)); // Output: true
console.log(isPalindrome(-121)); // Output: false

// ------------------------------------------------------------------------------------

// 3. Longest Common Prefix
// Write a function to find the longest common prefix string amongst an array of strings
// If there is no common prefix, return an empty string "".
// Example 1:
// Input: strs = ["flower","flow","flight"]
// Output: "fl"
// Example 2:
// Input: strs = ["dog","racecar","car"]
// Output: ""
// Explanation: There is no common prefix among the input strings.
var longestCommonPrefix = function (strs) {
  if (strs.length === 0) return "";
  let prefix = strs[0];
  console.log(`Initial prefix: ${prefix}`);
  for (let i = 1; i < strs.length; i++) {
    console.log(`Checking string: ${strs[i]}`);
    while (strs[i].indexOf(prefix) !== 0) {
      console.log(
        `Prefix not found in ${
          strs[i]
        }, shortening prefix from ${prefix}-----${strs[i].indexOf(prefix)}`
      );
      prefix = prefix.slice(0, -1);
      if (prefix === "") return "";
    }
  }
  return prefix;
};
console.log(longestCommonPrefix(["flower", "flow", "flight"])); // Output: "fl"
console.log(longestCommonPrefix(["dog", "racecar", "car"])); // Output: ""

// ------------------------------------------------------------------------------------

// 4. Valid Parentheses
// Given a string s containing just the characters '(', ')', '{', '}', '[' and ']',
// determine if the input string is valid.
// An input string is valid if:
```

```
401  // Open brackets must be closed by the same type of brackets.
402  // Open brackets must be closed in the correct order.
403  // Every close bracket has a corresponding open bracket of the same type.
404  // Example 1:
405  // Input: s = "()"
406  // Output: true
407  // Example 2:
408  // Input: s = "()[]{}"
409  // Output: true
410  // Example 3:
411  // Input: s = "(]"
412  // Output: false
413  // Example 4:
414  // Input: s = "([)]"
415  // Output: false
416  // Example 5:
417  // Input: s = "{[]}"
418  // Output: true
419  var isValid = function (s) {
420    const stack = [];
421    const map = {
422      "(": ")",
423      "{": "}",
424      "[": "]",
425    };
426    for (let char of s) {
427      if (map[char]) {
428        stack.push(map[char]);
429      } else if (stack.length === 0 || stack.pop() !== char) {
430        return false;
431      }
432    }
433    return stack.length === 0;
434  };
435  console.log(isValid("()")); // Output: true
436  console.log(isValid("()[]{}")); // Output: true
437  console.log(isValid("(]")); // Output: false
438  console.log(isValid("([)]")); // Output: false
439  console.log(isValid("{[]}")); // Output: true
440
441  // ---------------------------------------------------------------------------------
442
443  // 5. Remove Duplicates from Sorted Array
444  // Given a sorted array nums, remove the duplicates in-place such that each element appe
445  // Do not allocate extra space for another array, you must do this by modifying the inpu
446  // Example 1:
447  // Input: nums = [1,1,2]
448  // Output: 2, nums = [1,2]
449  // Explanation: Your function should return length = 2, with the first two elements of r
450  // It doesn't matter what you leave beyond the returned length.
451  // Example 2:
452  // Input: nums = [0,0,1,1,1,2,2,3,3,4]
```

```
453   // Output: 5, nums = [0,1,2,3,4]
454   var removeDuplicates = function (nums) {
455     if (nums.length === 0) return 0;
456     let i = 0;
457     for (let j = 1; j < nums.length; j++) {
458       console.log(`i: ${i}, j: ${j}, nums[i]: ${nums[i]}, nums[j]: ${nums[j]}`);
459
460       if (nums[i] !== nums[j]) {
461         i++;
462         nums[i] = nums[j];
463       }
464     }
465     return i + 1;
466   };
467   console.log(removeDuplicates([0, 0, 1, 1, 1, 2, 2, 3, 3, 4])); // Output: 5
468
469   // ------------------------------------------------------------------------------------
470
471   // 6. Largest word in a sentence
472   // Given a string s, return the length of the longest word in the sentence.
473   // A word is defined as a maximal substring consisting of non-space characters only.
474   // Example 1:
475   // Input: s = "The quick brown fox jumped over the lazy dog"
476   // Output: 6
477   // Explanation: The longest word in the sentence is "jumped", which has a length of 6.
478   var lengthOfLongestWord = function (s) {
479     const words = s.split(" ");
480     let maxLength = 0;
481     for (let word of words) {
482       console.log(`word: ${word}, length: ${word.length}`);
483       maxLength = Math.max(maxLength, word.length);
484     }
485     return maxLength;
486   };
487   console.log(
488     lengthOfLongestWord("The quick brown fox jumped over the lazy dog")
489   ); // Output: 6
490
491   // ------------------------------------------------------------------------------------
492
493   // FindUniqueValues in an Array
494   FindUniqueValues = function (arr) {
495     const uniqueValues = [];
496     const seen = new Set();
497     for (let i = 0; i < arr.length; i++) {
498       if (!seen.has(arr[i])) {
499         seen.add(arr[i]);
500         uniqueValues.push(arr[i]);
501       }
502     }
503     return uniqueValues;
504   };
```

```
505 console.log(FindUniqueValues([1, 2, 2, 3, 4, 4, 5])); // Output: [1, 2, 3, 4, 5]
506
507 // FindUniqueValues in an Array without using Set
508 FindUniqueValuesWithoutSet = function (arr) {
509   const uniqueValues = [];
510   for (let i = 0; i < arr.length; i++) {
511     if (!uniqueValues.includes(arr[i])) {
512       uniqueValues.push(arr[i]);
513     }
514   }
515   return uniqueValues;
516 };
517 console.log(FindUniqueValuesWithoutSet([1, 2, 2, 3, 4, 4, 5])); // Output: [1, 2, 3, 4,
518
519 // FindUniqueValues in an Array without using inbuilt function
520 const findUnique = (a) => {
521   let freq = {},
522     result = [];
523   for (let i = 0; i < a.length; i++) {
524     if (freq[a[i]] === 1) {
525       continue;
526     } else {
527       result.push(a[i]);
528       freq[a[i]] = 1;
529     }
530   }
531   return result;
532 };
533
534 const arr = [1, 1, 1, 2, 5, 2, 3, 8, 88, 88, 9, 9];
535 console.log(findUnique(arr));
536
537 // ------------------------------------------------------------------------------
538
539 // 4.       Program to reverse a string without using built-in methods.
540
541 function reverseString(str) {
542   let reversed = "";
543
544   // Loop from the end of the string to the beginning
545   for (let i = str.length - 1; i >= 0; i--) {
546     reversed += str[i];
547   }
548
549   return reversed;
550 }
551
552 // Example
553 console.log(reverseString("hello")); // Output: "olleh"
554 console.log(reverseString("JavaScript")); // Output: "tpircSavaJ"
555
556 // ------------------------------------------------------------------------------
```

```
557
558   // 5.       Find the maximum count of consecutive 1’s in an array.
559   function findMaxConsecutiveOnes(arr) {
560     let maxCount = 0;
561     let currentCount = 0;
562
563     for (let i = 0; i < arr.length; i++) {
564       if (arr[i] === 1) {
565         currentCount++;
566         maxCount = Math.max(maxCount, currentCount);
567       } else {
568         currentCount = 0;
569       }
570     }
571
572     return maxCount;
573   }
574
575   // Example
576   console.log(findMaxConsecutiveOnes([1, 1, 0, 1, 1, 1])); // Output: 3
577
578   // ----------------------------------------------------------------------------------
579
580   // 6.       Write a function to calculate the factorial of a given number.
581   // Iterative Approach
582   function factorialIterative(n) {
583     if (n < 0) return "Factorial not defined for negative numbers";
584
585     let result = 1;
586     for (let i = 1; i <= n; i++) {
587       result *= i;
588     }
589     return result;
590   }
591
592   // Example
593   console.log(factorialIterative(5)); // Output: 120
594   console.log(factorialIterative(0)); // Output: 1
595   // or
596   function factorialRecursive(n) {
597     if (n < 0) return "Factorial not defined for negative numbers";
598     if (n === 0 || n === 1) return 1;
599
600     return n * factorialRecursive(n - 1);
601   }
602
603   // Example
604   console.log(factorialRecursive(5)); // Output: 120
605   console.log(factorialRecursive(1)); // Output: 1
606
607   // ----------------------------------------------------------------------------------
608
```

```
609  // 7.       Merge and sort two sorted arrays. For example: [0, 3, 4, 31] and [4, 6, 30]
610  // ✅ Efficient Merge Approach (O(n + m))
611  function mergeSortedArrays(arr1, arr2) {
612    let i = 0,
613      j = 0;
614    let merged = [];
615
616    while (i < arr1.length && j < arr2.length) {
617      if (arr1[i] <= arr2[j]) {
618        merged.push(arr1[i]);
619        i++;
620      } else {
621        merged.push(arr2[j]);
622        j++;
623      }
624    }
625
626    // Add remaining elements
627    while (i < arr1.length) {
628      merged.push(arr1[i]);
629      i++;
630    }
631    while (j < arr2.length) {
632      merged.push(arr2[j]);
633      j++;
634    }
635
636    return merged;
637  }
638
639  // Example
640  console.log(mergeSortedArrays([0, 3, 4, 31], [4, 6, 30]));
641  // Output: [0, 3, 4, 4, 6, 30, 31]
642
643  // Simpler (but less efficient) Way
644  function mergeAndSort(arr1, arr2) {
645    return [...arr1, ...arr2].sort((a, b) => a - b);
646  }
647
648  console.log(mergeAndSort([0, 3, 4, 31], [4, 6, 30]));
649  // Output: [0, 3, 4, 4, 6, 30, 31]
650
651  // --------------------------------------------------------------------------------
652
653  // 8.       Check if every value in one array has a corresponding squared value in anoth
654  // ✅ Efficient Solution using Frequency Counters (O(n))
655  function same(arr1, arr2) {
656    if (arr1.length !== arr2.length) return false;
657
658    let freq1 = {};
659    let freq2 = {};
660
```

```
    // Count frequencies in arr1
    for (let num of arr1) {
      freq1[num] = (freq1[num] || 0) + 1;
    }

    // Count frequencies in arr2
    for (let num of arr2) {
      freq2[num] = (freq2[num] || 0) + 1;
    }

    // Compare
    for (let key in freq1) {
      let squared = key * key;

      if (!(squared in freq2)) return false; // squared value not present
      if (freq2[squared] !== freq1[key]) return false; // frequency mismatch
    }

    return true;
  }

  // Examples
  console.log(same([1, 2, 3], [1, 4, 9])); // true
  console.log(same([1, 2, 2], [1, 4, 4])); // true
  console.log(same([1, 2, 3], [1, 9])); // false
  console.log(same([1, 2, 3], [1, 4, 4])); // false

  // ✅ Simpler (but less efficient) Way
  function sameSimple(arr1, arr2) {
    if (arr1.length !== arr2.length) return false;

    for (let num of arr1) {
      let index = arr2.indexOf(num * num);
      if (index === -1) return false; // not found
      arr2.splice(index, 1); // remove matched element
    }
    return true;
  }

  console.log(sameSimple([1, 2, 3], [1, 4, 9])); // true

  // 9.       Check if one string can be rearranged to form another.
  // ✅ Efficient Approach (Frequency Counter)
  function isAnagram(str1, str2) {
    if (str1.length !== str2.length) return false;

    let freq1 = {};
    let freq2 = {};

    for (let char of str1) {
      freq1[char] = (freq1[char] || 0) + 1;
    }
```

```
713
714       for (let char of str2) {
715         freq2[char] = (freq2[char] || 0) + 1;
716       }
717
718       for (let key in freq1) {
719         if (freq1[key] !== freq2[key]) return false;
720       }
721
722       return true;
723     }
724
725     // Examples
726     console.log(isAnagram("listen", "silent")); // true
727     console.log(isAnagram("triangle", "integral")); // true
728     console.log(isAnagram("hello", "bello")); // false
729
730     // ✅ Optimized Version (Single Frequency Counter)
731     function isAnagramOptimized(str1, str2) {
732       if (str1.length !== str2.length) return false;
733
734       let lookup = {};
735
736       for (let char of str1) {
737         lookup[char] = (lookup[char] || 0) + 1;
738       }
739
740       for (let char of str2) {
741         if (!lookup[char]) return false; // char not found or freq mismatch
742         lookup[char] -= 1;
743       }
744
745       return true;
746     }
747
748     // Examples
749     console.log(isAnagramOptimized("listen", "silent")); // true
750     console.log(isAnagramOptimized("rat", "car")); // false
751
752     // 10.   Extract unique objects from an array of objects.
753     // i/o- const arr = [
754     //   { id: 1, name: "Alice" },
755     //   { id: 2, name: "Bob" },
756     //   { id: 1, name: "Alice" }, // duplicate
757     //   { id: 3, name: "Charlie" }
758     // ];
759     // o/p- [
760     //   { id: 1, name: "Alice" },
761     //   { id: 2, name: "Bob" },
762     //   { id: 3, name: "Charlie" }
763     // ]
764
```

```
765 function getUniqueObjects(arr, key) {
766   return arr.reduce((acc, curr) => {
767     if (!acc.find((item) => item[key] === curr[key])) {
768       acc.push(curr);
769     }
770     return acc;
771   }, []);
772 }
773
774 console.log(getUniqueObjects(arr, "id"));
775 // ------------------------------------------
776 function getUniqueObjects(arr, key) {
777   const map = new Map();
778   for (let obj of arr) {
779     map.set(obj[key], obj); // overwrite duplicates
780   }
781   return Array.from(map.values());
782 }
783
784 // Example
785 const unique = getUniqueObjects(arr, "id");
786 console.log(unique);
787
788 // 11.   Write a program to find the maximum number in an array.
789 function findMax(arr) {
790   if (arr.length === 0) return undefined; // handle empty array
791
792   let max = arr[0]; // assume first element is max
793   for (let i = 1; i < arr.length; i++) {
794     if (arr[i] > max) {
795       max = arr[i];
796     }
797   }
798   return max;
799 }
800
801 // Example
802 console.log(findMax([10, 3, 45, 7, 99, 23])); // 99
803
804 // ------------------------------------------------
805
806 function findMax(arr) {
807   return arr.length ? Math.max(...arr) : undefined;
808 }
809
810 console.log(findMax([10, 3, 45, 7, 99, 23])); // 99
811
812 // ------------------------------------------------
813
814 function findMax(arr) {
815   return arr.reduce((max, num) => (num > max ? num : max), arr[0]);
816 }
```

```
817
818     console.log(findMax([10, 3, 45, 7, 99, 23])); // 99
819
820     // ------------------------------------------------
821
822     // 12.   Filter and return only even numbers from an array.
823     function getEvenNumbers(arr) {
824       return arr.filter(num => num % 2 === 0);
825     }
826
827     // Example
828     console.log(getEvenNumbers([1, 2, 3, 4, 5, 6, 7, 8]));
829     // Output: [2, 4, 6, 8]
830
831     // ------------------------------------------------------------------
832     function getEvenNumbers(arr) {
833       let evens = [];
834       for (let i = 0; i < arr.length; i++) {
835         if (arr[i] % 2 === 0) {
836           evens.push(arr[i]);
837         }
838       }
839       return evens;
840     }
841
842     // Example
843     console.log(getEvenNumbers([1, 2, 3, 4, 5, 6, 7, 8]));
844     // Output: [2, 4, 6, 8]
845     // ------------------------------------------------------------------
846     function getEvenNumbers(arr) {
847       return arr.reduce((evens, num) => {
848         if (num % 2 === 0) evens.push(num);
849         return evens;
850       }, []);
851     }
852
853     // Example
854     console.log(getEvenNumbers([1, 2, 3, 4, 5, 6, 7, 8]));
855     // Output: [2, 4, 6, 8]
856
857
858     // 13.   Check if a number is prime.
859     const isPrime = num =>
860       num > 1 && [...Array(Math.floor(Math.sqrt(num)) + 1).keys()]
861         .slice(2)
862         .every(i => num % i !== 0);
863
864     console.log(isPrime(7));  // true
865     console.log(isPrime(25)); // false
866     // ------------------------
867     function isPrime(num) {
868       if (num <= 1) return false;
```

```
869       if (num === 2) return true;
870       if (num % 2 === 0) return false; // even numbers > 2 are not prime
871
872       for (let i = 3; i <= Math.sqrt(num); i += 2) {
873         if (num % i === 0) return false;
874       }
875       return true;
876     }
877
878     // Example
879     console.log(isPrime(29)); // true
880     console.log(isPrime(30)); // false
881     // ---------------------------------------
882     function isPrime(num) {
883       if (num <= 1) return false; // 0 and 1 are not prime
884       for (let i = 2; i < num; i++) {
885         if (num % i === 0) return false; // divisible by something other than 1 & itself
886       }
887       return true;
888     }
889
890     // Example
891     console.log(isPrime(2));  // true
892     console.log(isPrime(17)); // true
893     console.log(isPrime(18)); // false
894
895
896     // ------------------------------------------------------------------------------------
897
898     // 14.   Find the largest number in a nested array.
899     // const arr = [[3, 5, 9], [1, 4, 7], [2, 8, 6]];
900     // 9
901     function findLargestNested(arr) {
902       const flat = arr.flat(Infinity); // flattens nested arrays
903       return Math.max(...flat);
904     }
905
906     // Example
907     console.log(findLargestNested([[3, 5, 9], [1, 4, 7], [2, 8, 6]]));
908     // Output: 9
909     // ---------------------------------------------------------------
910     function findLargestNested(arr) {
911       let max = -Infinity;
912
913       for (let subArr of arr) {
914         for (let num of subArr) {
915           if (num > max) {
916             max = num;
917           }
918         }
919       }
920
```

```
921       return max;
922     }
923
924     // Example
925     console.log(findLargestNested([[3, 5, 9], [1, 4, 7], [2, 8, 6]]));
926     // Output: 9
927     // -----------------------------------------
928     function findLargestNested(arr) {
929       let max = -Infinity;
930
931       function helper(subArr) {
932         for (let el of subArr) {
933           if (Array.isArray(el)) {
934             helper(el);
935           } else if (el > max) {
936             max = el;
937           }
938         }
939       }
940
941       helper(arr);
942       return max;
943     }
944
945     // Example
946     console.log(findLargestNested([[3, [15, 2]], [1, [9, [22]]], [7, 8]]));
947     // Output: 22
948     // ---------------------------------------------------------------------------------
949     // 15.   Generate a Fibonacci sequence up to a specified number of terms.
950     function fibonacci(n) {
951       if (n <= 0) return [];
952       if (n === 1) return [0];
953
954       let seq = [0, 1];
955       for (let i = 2; i < n; i++) {
956         seq.push(seq[i - 1] + seq[i - 2]);
957       }
958       return seq;
959     }
960
961     // Example
962     console.log(fibonacci(10));
963     // Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
964     // -----------------------------------------
965     function fibonacciRecursive(n) {
966       if (n <= 1) return n;
967       return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
968     }
969
970     function generateFibonacci(n) {
971       let seq = [];
972       for (let i = 0; i < n; i++) {
```

```
973         seq.push(fibonacciRecursive(i));
974       }
975       return seq;
976     }
977
978     // Example
979     console.log(generateFibonacci(10));
980     // Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
981     // ------------------------------------------
982     function fibonacci(n) {
983       let seq = [];
984       let a = 0, b = 1;
985       let count = 0;
986
987       while (count < n) {
988         seq.push(a);
989         [a, b] = [b, a + b]; // swap trick
990         count++;
991       }
992
993       return seq;
994     }
995
996     // Example
997     console.log(fibonacci(10));
998     // Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
999     // ------------------------------------------------------------------
1000    // 16.   Count the occurrences of each character in a string.
1001    function charCount(str) {
1002      let count = {};
1003      for (let char of str) {
1004        count[char] = (count[char] || 0) + 1;
1005      }
1006      return count;
1007    }
1008
1009    // Example
1010    console.log(charCount("hello"));
1011    // Output: { h: 1, e: 1, l: 2, o: 1 }
1012    // ---------------------------------------
1013    function charCount(str) {
1014      return str.split("").reduce((acc, char) => {
1015        acc[char] = (acc[char] || 0) + 1;
1016        return acc;
1017      }, {});
1018    }
1019
1020    // Example
1021    console.log(charCount("banana"));
1022    // Output: { b: 1, a: 3, n: 2 }
1023    // ----------------------------------------
1024    function charCount(str) {
```

```
  let count = {};
  for (let char of str.toLowerCase()) {
    if (/[a-z0-9]/.test(char)) { // only count letters/numbers
      count[char] = (count[char] || 0) + 1;
    }
  }
  return count;
}

// Example
console.log(charCount("Hello World 123!"));
// Output: { h: 1, e: 1, l: 3, o: 2, w: 1, r: 1, d: 1, '1': 1, '2': 1, '3': 1 }
// ------------------------------------------------------------------------------------
// 17.   Sort an array of numbers in ascending order.
function sortArray(arr) {
  return arr.sort((a, b) => a - b);
}

// Example
console.log(sortArray([5, 2, 9, 1, 7]));
// Output: [1, 2, 5, 7, 9]
// ----------------------------------------
function bubbleSort(arr) {
  let n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    for (let j = 0; j < n - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        // swap
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
      }
    }
  }
  return arr;
}

// Example
console.log(bubbleSort([5, 2, 9, 1, 7]));
// Output: [1, 2, 5, 7, 9]
// ----------------------------------------
function sortArray(arr) {
  return arr.reduce((sorted, num) => {
    let i = 0;
    while (i < sorted.length && num > sorted[i]) {
      i++;
    }
    sorted.splice(i, 0, num);
    return sorted;
  }, []);
}

// Example
console.log(sortArray([5, 2, 9, 1, 7]));
```

```
// Output: [1, 2, 5, 7, 9]
// ------------------------------------------------------------------------------------
// 18.   Sort an array of numbers in descending order.
function sortDescending(arr) {
  return arr.sort((a, b) => b - a);
}

// Example
console.log(sortDescending([5, 2, 9, 1, 7]));
// Output: [9, 7, 5, 2, 1]
// -------------------------------------
function bubbleSortDescending(arr) {
  let n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    for (let j = 0; j < n - i - 1; j++) {
      if (arr[j] < arr[j + 1]) {
        // swap
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
      }
    }
  }
  return arr;
}

// Example
console.log(bubbleSortDescending([5, 2, 9, 1, 7]));
// Output: [9, 7, 5, 2, 1]
// -----------------------------------
function sortDescending(arr) {
  return arr.sort((a, b) => a - b).reverse();
}

// Example
console.log(sortDescending([5, 2, 9, 1, 7]));
// Output: [9, 7, 5, 2, 1]
// ------------------------------------------------------------------------------------
// 19.   Reverse the order of words in a sentence without using the reverse() method.
function reverseWords(sentence) {
  let words = sentence.split(" ");
  let result = "";

  for (let i = words.length - 1; i >= 0; i--) {
    result += words[i];
    if (i > 0) result += " "; // add space except for last word
  }

  return result;
}

// Example
console.log(reverseWords("I love JavaScript"));
// Output: "JavaScript love I"
```

```
1129  // --------------------------------
1130  function reverseWords(sentence) {
1131    let words = sentence.split(" ");
1132    let reversed = [];
1133
1134    for (let i = 0; i < words.length; i++) {
1135      reversed.unshift(words[i]); // insert at front
1136    }
1137
1138    return reversed.join(" ");
1139  }
1140
1141  // Example
1142  console.log(reverseWords("Learning JavaScript is fun"));
1143  // Output: "fun is JavaScript Learning"
1144  // --------------------------------
1145  function reverseWords(sentence) {
1146    let word = "";
1147    let words = [];
1148
1149    for (let i = 0; i < sentence.length; i++) {
1150      if (sentence[i] === " ") {
1151        words.push(word);
1152        word = "";
1153      } else {
1154        word += sentence[i];
1155      }
1156    }
1157    words.push(word); // push last word
1158
1159    // build reversed sentence
1160    let result = "";
1161    for (let i = words.length - 1; i >= 0; i--) {
1162      result += words[i];
1163      if (i > 0) result += " ";
1164    }
1165    return result;
1166  }
1167
1168  // Example
1169  console.log(reverseWords("Hello world from JS"));
1170  // Output: "JS from world Hello"
1171  // ------------------------------------------------------------------------------------
1172  // 20.   Flatten a deeply nested array into a single-dimensional array.
1173  const arr = [1, [2, [3, [4, [5]]]], 6];
1174  console.log(arr.flat(Infinity));
1175  // Output: [1, 2, 3, 4, 5, 6]
1176  // --------------------------
1177  function flattenArray(arr) {
1178    let result = [];
1179    let stack = [...arr];
1180
```

```
1181      while (stack.length) {
1182        let next = stack.pop();
1183        if (Array.isArray(next)) {
1184          stack.push(...next); // expand nested arrays
1185        } else {
1186          result.push(next);
1187        }
1188      }
1189
1190      return result.reverse(); // because stack reverses order
1191    }
1192
1193    // Example
1194    console.log(flattenArray([1, [2, [3, [4, [5]]]], 6]));
1195    // Output: [1, 2, 3, 4, 5, 6]
1196    // ----------------------------------------
1197    function flattenArray(arr) {
1198      return arr.reduce((acc, val) =>
1199        acc.concat(Array.isArray(val) ? flattenArray(val) : val), []
1200      );
1201    }
1202
1203    // Example
1204    console.log(flattenArray([1, [2, [3, [4, [5]]]], 6]));
1205    // Output: [1, 2, 3, 4, 5, 6]
1206    // --------------------------------
1207    function flattenArray(arr) {
1208      let result = [];
1209
1210      for (let el of arr) {
1211        if (Array.isArray(el)) {
1212          result = result.concat(flattenArray(el)); // recurse if nested
1213        } else {
1214          result.push(el);
1215        }
1216      }
1217
1218      return result;
1219    }
1220
1221    // Example
1222    console.log(flattenArray([1, [2, [3, [4, [5]]]], 6]));
1223    // Output: [1, 2, 3, 4, 5, 6]
1224    // ------------------------------------------------------------------------------------
1225    // 21.   Convert a string input into a nested object (e.g., "a.b.c", "someValue" should
1226    function stringToNestedObject(path, value) {
1227      const keys = path.split(".");
1228      let result = {};
1229      let current = result;
1230
1231      for (let i = 0; i < keys.length; i++) {
1232        if (i === keys.length - 1) {
```

```
1233          current[keys[i]] = value; // last key gets the value
1234        } else {
1235          current[keys[i]] = {};
1236          current = current[keys[i]];
1237        }
1238      }
1239
1240      return result;
1241    }
1242
1243    // Example
1244    console.log(stringToNestedObject("a.b.c", "someValue"));
1245    // Output: { a: { b: { c: "someValue" } } }
1246    // ------------------------------------------
1247    function stringToNestedObject(path, value) {
1248      return path
1249        .split(".")
1250        .reduceRight((acc, key) => ({ [key]: acc }), value);
1251    }
1252
1253    // Example
1254    console.log(stringToNestedObject("user.profile.name", "Alice"));
1255    // Output: { user: { profile: { name: "Alice" } } }
1256    // ------------------------------------------
1257    function stringToNestedObject(path, value) {
1258      const keys = path.split(".");
1259
1260      function build(keys, value) {
1261        if (keys.length === 0) return value;
1262        const [first, ...rest] = keys;
1263        return { [first]: build(rest, value) };
1264      }
1265
1266      return build(keys, value);
1267    }
1268
1269    // Example
1270    console.log(stringToNestedObject("x.y.z", 42));
1271    // Output: { x: { y: { z: 42 } } }
1272    // ------------------------------------------------------------------------------------
1273    // 22.   Write a function that converts an object into a query string (e.g., {name: "Joh
1274    function toQueryString(obj) {
1275      return Object.entries(obj)
1276        .map(([key, value]) => `${encodeURIComponent(key)}=${encodeURIComponent(value)}`)
1277        .join("&");
1278    }
1279
1280    // Example
1281    console.log(toQueryString({ name: "John", age: 30 }));
1282    // Output: "name=John&age=30"
1283    // ---------------------------------
1284    function toQueryString(obj) {
```

```
    let query = [];
    for (let key in obj) {
      if (obj.hasOwnProperty(key)) {
        query.push(encodeURIComponent(key) + "=" + encodeURIComponent(obj[key]));
      }
    }
    return query.join("&");
  }

  // Example
  console.log(toQueryString({ city: "New York", job: "Developer" }));
  // Output: "city=New%20York&job=Developer"
  // ----------------------------------------
  function toQueryString(obj) {
    return new URLSearchParams(obj).toString();
  }

  // Example
  console.log(toQueryString({ name: "John", age: 30 }));
  // Output: "name=John&age=30"
  // ------------------------------------------------------------------------------------------
  // 23.   Implement a function to deep clone an object.
  function deepClone(obj) {
    if (obj === null || typeof obj !== "object") {
      return obj; // return primitives as is
    }

    if (Array.isArray(obj)) {
      return obj.map(item => deepClone(item)); // clone array elements
    }

    let clonedObj = {};
    for (let key in obj) {
      if (obj.hasOwnProperty(key)) {
        clonedObj[key] = deepClone(obj[key]); // recurse
      }
    }
    return clonedObj;
  }

  // Example
  const original = { a: 1, b: { c: 2, d: [3, 4] } };
  const copy = deepClone(original);
  copy.b.c = 99;

  console.log(original.b.c); // ✅ 2 (unchanged)
  console.log(copy.b.c);     // ✅ 99
  // ------------------------------------------------------------------------------------------

  // 24.   Write a function to find the index of the first non-repeating character in a st
  function firstNonRepeatingCharIndex(str) {
      const charCount = {};
```

```
1337
1338        // Count the occurrences of each character
1339        for (let char of str) {
1340            charCount[char] = (charCount[char] || 0) + 1;
1341        }
1342
1343        // Find the index of the first character with count 1
1344        for (let i = 0; i < str.length; i++) {
1345            if (charCount[str[i]] === 1) {
1346                return i;
1347            }
1348        }
1349
1350        // Return -1 if no non-repeating character is found
1351        return -1;
1352    }
1353
1354    // Example usage:
1355    console.log(firstNonRepeatingCharIndex("leetcode")); // Output: 0 ('l')
1356    console.log(firstNonRepeatingCharIndex("loveleetcode")); // Output: 2 ('v')
1357    console.log(firstNonRepeatingCharIndex("aabb")); // Output: -1 (no non-repeating char)
1358    // ------------------------------------------------------------------------------------
1359    // 25.   Check if a number is an Armstrong number (e.g., 153 = 1³ + 5³ + 3³).
1360    function isArmstrongNumber(num) {
1361      const digits = num.toString().split('').map(Number);
1362      const power = digits.length;
1363
1364      const sum = digits.reduce((acc, digit) => acc + Math.pow(digit, power), 0);
1365
1366      return sum === num;
1367    }
1368
1369    // Examples
1370    console.log(isArmstrongNumber(153));   // true
1371    console.log(isArmstrongNumber(9474));  // true
1372    console.log(isArmstrongNumber(123));   // false
1373
1374    // ------------------------------------------------------------------------------------
1375
1376
1377    // 1️⃣ Reverse a string
1378    function reverseString(str) {
1379        let reversed = '';
1380        for (let i = str.length - 1; i >= 0; i--) {
1381            reversed += str[i];
1382        }
1383        return reversed;
1384    }
1385
1386    // Example
1387    console.log(reverseString("hello")); // "olleh"
1388
```

```
// 2️⃣ Check palindrome string
function isPalindrome(str) {
    let start = 0;
    let end = str.length - 1;
    while (start < end) {
        if (str[start] !== str[end]) return false;
        start++;
        end--;
    }
    return true;
}

// Example
console.log(isPalindrome("level")); // true
console.log(isPalindrome("hello")); // false

// 3️⃣ Count vowels in a string
function countVowels(str) {
    let count = 0;
    for (let i = 0; i < str.length; i++) {
        let ch = str[i];
        if (ch === 'a' || ch === 'e' || ch === 'i' || ch === 'o' || ch === 'u' ||
            ch === 'A' || ch === 'E' || ch === 'I' || ch === 'O' || ch === 'U') {
            count++;
        }
    }
    return count;
}

// Example
console.log(countVowels("Hello World")); // 3

// 4️⃣ Find first non-repeated character
function firstNonRepeatedChar(str) {
    for (let i = 0; i < str.length; i++) {
        let repeated = false;
        for (let j = 0; j < str.length; j++) {
            if (i !== j && str[i] === str[j]) {
                repeated = true;
                break;
            }
        }
        if (!repeated) return str[i];
    }
    return null;
}

// Example
console.log(firstNonRepeatedChar("swiss")); // "w"

// 5️⃣ Check if two strings are anagrams
function areAnagrams(str1, str2) {
```

```
1441        if (str1.length !== str2.length) return false;
1442
1443        // Count characters in str1
1444        let count1 = {};
1445        let count2 = {};
1446
1447        for (let i = 0; i < str1.length; i++) {
1448            count1[str1[i]] = (count1[str1[i]] || 0) + 1;
1449            count2[str2[i]] = (count2[str2[i]] || 0) + 1;
1450        }
1451
1452        for (let ch in count1) {
1453            if (count1[ch] !== count2[ch]) return false;
1454        }
1455
1456        return true;
1457    }
1458
1459    // Example
1460    console.log(areAnagrams("listen", "silent")); // true
1461    console.log(areAnagrams("hello", "world")); // false
1462
1463    // 6️⃣ Capitalize first letter of each word
1464    function capitalizeWords(str) {
1465        let result = '';
1466        let capitalizeNext = true;
1467
1468        for (let i = 0; i < str.length; i++) {
1469            if (str[i] === ' ') {
1470                result += ' ';
1471                capitalizeNext = true;
1472            } else {
1473                if (capitalizeNext) {
1474                    // Convert lowercase to uppercase manually
1475                    let code = str[i].charCodeAt(0);
1476                    if (code >= 97 && code <= 122) {
1477                        result += String.fromCharCode(code - 32);
1478                    } else {
1479                        result += str[i];
1480                    }
1481                    capitalizeNext = false;
1482                } else {
1483                    result += str[i];
1484                }
1485            }
1486        }
1487        return result;
1488    }
1489
1490    // Example
1491    console.log(capitalizeWords("hello world from javascript")); // "Hello World From Javasc
1492
```

```
1493    // 7️⃣ Remove duplicates from a string
1494    function removeDuplicates(str) {
1495        let result = '';
1496        for (let i = 0; i < str.length; i++) {
1497            let exists = false;
1498            for (let j = 0; j < result.length; j++) {
1499                if (str[i] === result[j]) {
1500                    exists = true;
1501                    break;
1502                }
1503            }
1504            if (!exists) result += str[i];
1505        }
1506        return result;
1507    }
1508
1509    // Example
1510    console.log(removeDuplicates("programming")); // "progamin"
1511
1512    // ------------------------------------------------------------------------
1513
1514
1515
1516    // 1️⃣ Find max and min in an array
1517    function findMaxMin(arr) {
1518        if (arr.length === 0) return null;
1519
1520        let max = arr[0];
1521        let min = arr[0];
1522
1523        for (let i = 1; i < arr.length; i++) {
1524            if (arr[i] > max) max = arr[i];
1525            if (arr[i] < min) min = arr[i];
1526        }
1527
1528        return { max, min };
1529    }
1530
1531    // Example
1532    console.log(findMaxMin([3, 7, 1, 9, 2])); // { max: 9, min: 1 }
1533
1534    // 2️⃣ Reverse an array without reverse()
1535    function reverseArray(arr) {
1536        let reversed = [];
1537        for (let i = arr.length - 1; i >= 0; i--) {
1538            reversed[reversed.length] = arr[i];
1539        }
1540        return reversed;
1541    }
1542
1543    // Example
1544    console.log(reverseArray([1, 2, 3, 4])); // [4, 3, 2, 1]
```

```
1545
1546    // 3️⃣ Check if array contains duplicates
1547    function hasDuplicates(arr) {
1548        for (let i = 0; i < arr.length; i++) {
1549            for (let j = i + 1; j < arr.length; j++) {
1550                if (arr[i] === arr[j]) return true;
1551            }
1552        }
1553        return false;
1554    }
1555
1556    // Example
1557    console.log(hasDuplicates([1, 2, 3, 2])); // true
1558    console.log(hasDuplicates([1, 2, 3]));    // false
1559
1560    // 4️⃣ Find second largest number in an array
1561    function secondLargest(arr) {
1562        if (arr.length < 2) return null;
1563
1564        let largest = -Infinity;
1565        let second = -Infinity;
1566
1567        for (let i = 0; i < arr.length; i++) {
1568            if (arr[i] > largest) {
1569                second = largest;
1570                largest = arr[i];
1571            } else if (arr[i] > second && arr[i] !== largest) {
1572                second = arr[i];
1573            }
1574        }
1575        return second;
1576    }
1577
1578    // Example
1579    console.log(secondLargest([5, 3, 9, 7, 9])); // 7
1580
1581    // 5️⃣ Find intersection of two arrays
1582    function intersection(arr1, arr2) {
1583        let result = [];
1584        for (let i = 0; i < arr1.length; i++) {
1585            for (let j = 0; j < arr2.length; j++) {
1586                if (arr1[i] === arr2[j]) {
1587                    let exists = false;
1588                    for (let k = 0; k < result.length; k++) {
1589                        if (result[k] === arr1[i]) {
1590                            exists = true;
1591                            break;
1592                        }
1593                    }
1594                    if (!exists) result[result.length] = arr1[i];
1595                }
1596            }
```

```
    }
    return result;
}

// Example
console.log(intersection([1,2,3,4], [3,4,5,6])); // [3,4]

// 6️⃣ Flatten a nested array
function flattenArray(arr) {
    let result = [];
    for (let i = 0; i < arr.length; i++) {
        if (arr[i] instanceof Array) {
            let flat = flattenArray(arr[i]);
            for (let j = 0; j < flat.length; j++) {
                result[result.length] = flat[j];
            }
        } else {
            result[result.length] = arr[i];
        }
    }
    return result;
}

// Example
console.log(flattenArray([1, [2, [3, 4], 5], 6])); // [1, 2, 3, 4, 5, 6]

// 7️⃣ Rotate array by k steps
function rotateArray(arr, k) {
    let n = arr.length;
    k = k % n;
    let rotated = [];

    for (let i = 0; i < n; i++) {
        let newIndex = (i + k) % n;
        rotated[newIndex] = arr[i];
    }

    return rotated;
}

// Example
console.log(rotateArray([1,2,3,4,5], 2)); // [4,5,1,2,3]

// 8️⃣ Remove falsy values from an array
function removeFalsy(arr) {
    let result = [];
    for (let i = 0; i < arr.length; i++) {
        if (arr[i]) result[result.length] = arr[i];
    }
    return result;
}

```

```
1649    // Example
1650    console.log(removeFalsy([0, 1, false, 2, "", 3, null])); // [1, 2, 3]
1651
1652    // ------------------------------------------------------------------------------------
1653
1654    // Number & Logic
1655    // 1️⃣ Check prime number
1656    function isPrime(num) {
1657        if (num <= 1) return false;
1658        for (let i = 2; i * i <= num; i++) {
1659            if (num % i === 0) return false;
1660        }
1661        return true;
1662    }
1663
1664    // Example
1665    console.log(isPrime(7)); // true
1666    console.log(isPrime(10)); // false
1667
1668    // 2️⃣ Generate Fibonacci series
1669    function fibonacciSeries(n) {
1670        let series = [];
1671        if (n >= 1) series[0] = 0;
1672        if (n >= 2) series[1] = 1;
1673
1674        for (let i = 2; i < n; i++) {
1675            series[i] = series[i - 1] + series[i - 2];
1676        }
1677        return series;
1678    }
1679
1680    // Example
1681    console.log(fibonacciSeries(7)); // [0, 1, 1, 2, 3, 5, 8]
1682
1683    // 3️⃣ Find factorial of a number
1684    function factorial(n) {
1685        if (n < 0) return null; // Factorial not defined for negative numbers
1686        let fact = 1;
1687        for (let i = 2; i <= n; i++) {
1688            fact *= i;
1689        }
1690        return fact;
1691    }
1692
1693    // Example
1694    console.log(factorial(5)); // 120
1695
1696    // 4️⃣ Find sum of digits of a number
1697    function sumOfDigits(num) {
1698        let sum = 0;
1699        if (num < 0) num = -num; // handle negative numbers
1700        while (num > 0) {
```

```
1701            sum += num % 10;
1702            num = Math.floor(num / 10);
1703        }
1704        return sum;
1705    }
1706
1707    // Example
1708    console.log(sumOfDigits(1234)); // 10
1709
1710    // 5️⃣ Check Armstrong number
1711
1712    // (A number is Armstrong if the sum of its digits each raised to the power of number of
1713
1714    function isArmstrong(num) {
1715        let original = num;
1716        let sum = 0;
1717        let digits = 0;
1718        let temp = num;
1719
1720        // Count digits
1721        while (temp > 0) {
1722            digits++;
1723            temp = Math.floor(temp / 10);
1724        }
1725
1726        temp = num;
1727        while (temp > 0) {
1728            let digit = temp % 10;
1729            let power = 1;
1730            for (let i = 0; i < digits; i++) power *= digit; // manual power
1731            sum += power;
1732            temp = Math.floor(temp / 10);
1733        }
1734
1735        return sum === original;
1736    }
1737
1738    // Example
1739    console.log(isArmstrong(153)); // true
1740    console.log(isArmstrong(123)); // false
1741
1742    // 6️⃣ Find GCD of two numbers
1743    function gcd(a, b) {
1744        while (b !== 0) {
1745            let temp = b;
1746            b = a % b;
1747            a = temp;
1748        }
1749        return a;
1750    }
1751
1752    // Example
```

```
1753    // console.log(gcd(12, 18)); // 6
1754
1755
1756    // 1️⃣ Count frequency of elements in an array
1757    function countFrequency(arr) {
1758        let freq = {};
1759        for (let i = 0; i < arr.length; i++) {
1760            let found = false;
1761            for (let key in freq) {
1762                if (key == arr[i]) {
1763                    freq[key]++;
1764                    found = true;
1765                    break;
1766                }
1767            }
1768            if (!found) freq[arr[i]] = 1;
1769        }
1770        return freq;
1771    }
1772
1773    // Example
1774    console.log(countFrequency([1,2,2,3,3,3])); // { '1': 1, '2': 2, '3': 3 }
1775
1776    // 2️⃣ Group objects by a property
1777    function groupBy(arr, key) {
1778        let result = {};
1779        for (let i = 0; i < arr.length; i++) {
1780            let prop = arr[i][key];
1781            let exists = false;
1782            for (let k in result) {
1783                if (k == prop) {
1784                    result[k][result[k].length] = arr[i];
1785                    exists = true;
1786                    break;
1787                }
1788            }
1789            if (!exists) result[prop] = [arr[i]];
1790        }
1791        return result;
1792    }
1793
1794    // Example
1795    let data = [
1796        {name: "Alice", role: "admin"},
1797        {name: "Bob", role: "user"},
1798        {name: "Charlie", role: "admin"}
1799    ];
1800    console.log(groupBy(data, "role"));
1801    // { admin: [{…}, {…}], user: [{…}] }
1802
1803    // 3️⃣ Deep clone an object
1804    function deepClone(obj) {
```

```
1805        if (obj === null || typeof obj !== 'object') return obj;
1806
1807        let clone = obj instanceof Array ? [] : {};
1808
1809        for (let key in obj) {
1810            if (obj.hasOwnProperty(key)) {
1811                clone[key] = deepClone(obj[key]);
1812            }
1813        }
1814        return clone;
1815    }
1816
1817    // Example
1818    let original = {a: 1, b: {c: 2}};
1819    let cloned = deepClone(original);
1820    cloned.b.c = 5;
1821    console.log(original.b.c); // 2 (unchanged)
1822
1823    // 4️⃣ Implement debounce function
1824
1825    // (Debounce delays function execution until a certain time has passed without it being
1826
1827    function debounce(func, delay) {
1828        let timer = null;
1829        return function() {
1830            let context = this;
1831            let args = [];
1832            for (let i = 0; i < arguments.length; i++) args[i] = arguments[i];
1833
1834            if (timer) clearTimeout(timer);
1835
1836            timer = setTimeout(function() {
1837                func.apply(context, args);
1838            }, delay);
1839        };
1840    }
1841
1842    // Example
1843    function sayHello(name) {
1844        console.log("Hello, " + name);
1845    }
1846
1847    let debouncedHello = debounce(sayHello, 1000);
1848    debouncedHello("Alice");
1849    debouncedHello("Bob"); // Only "Hello, Bob" will be printed after 1 sec
1850
1851
1852
1853    //  ------------------------------------------------------------------------
1854
1855
1856
```

```
1857
1858
1859    //Left Rotate an Array by One Position
1860    let arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
1861    let copy = arr[0];
1862
1863    for (let i = 0; i < arr.length - 1; i++) {
1864      arr[i] = arr[i + 1];
1865    }
1866    arr[arr.length - 1] = copy;
1867    console.log(arr); // Output: [2, 3, 4, 5, 6, 7, 8, 9, 10, 1]
1868
1869    //right Rotate an Array by One Position
1870    let arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
1871    let copy = arr[arr.length - 1];
1872    for (let i = arr.length - 1; i > 0; i--) {
1873      arr[i] = arr[i - 1];
1874    }
1875    arr[0] = copy;
1876    console.log(arr); // Output: [10, 1, 2, 3, 4, 5, 6, 7, 8, 9]
1877
1878    // Left Rotate an Array by K Positions
1879    function leftRotate(arr, k) {
1880      k = k % arr.length;
1881      count = 0;
1882      for (let j = 0; j < k; j++) {
1883        count++;
1884        let copy = arr[0];
1885        for (let i = 0; i < arr.length - 1; i++) {
1886          arr[i] = arr[i + 1];
1887        }
1888        arr[arr.length - 1] = copy;
1889      }
1890      return arr;
1891    }
1892
1893    // or
1894
1895    function leftArry(arr, k) {
1896      let temp = new Array(arr.length);
1897      k = k % arr.length;
1898      for (let i = 0; i < arr.length; i++) {
1899        temp[i] = arr[(i + k) % arr.length];
1900      }
1901      return temp;
1902    }
1903
1904    // best optimization or
1905    let arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
1906    let k = Number(prompt("Enter the number of positions to rotate:"));
1907    k = k % arr.length;
1908    reverse(0, k - 1);
```

```
1909    reverse(k, arr.length - 1);
1910    reverse(0, arr.length - 1);
1911    console.log(arr);
1912
1913    function reverse(i, j) {
1914      while (i < j) {
1915        let temp = arr[i];
1916        arr[i] = arr[j];
1917        arr[j] = temp;
1918        i++;
1919        j--;
1920      }
1921    }
1922    // // Example usage:
1923    let k = 1;
1924    console.log(leftRotate(arr, k));
1925    console.log(count);
1926
1927    // ---------------------------------------------------
1928
1929    // Right Rotate an Array by K Positions
1930    function rightArry(arr, k) {
1931      let temp = new Array(arr.length);
1932      k = k % arr.length; // normalize k
1933      for (let i = 0; i < arr.length; i++) {
1934        temp[(i + k) % arr.length] = arr[i];
1935      }
1936      return temp;
1937    }
1938
1939    let arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
1940    let k = 3;
1941    console.log(rightArry(arr, k));
1942
1943    function rightRotate(arr, k) {
1944      k = k % arr.length;
1945      for (let j = 0; j < k; j++) {
1946          let copy = arr[arr.length - 1];
1947          for (let i = arr.length - 1; i > 0; i--) {
1948              arr[i] = arr[i - 1];
1949        }
1950        arr[0] = copy;
1951      }
1952      return arr;
1953    }
1954    // Example usage:
1955    let arr2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
1956    let k2 = 3;
1957    console.log(rightRotate(arr2, k2));
1958
1959    // -----------------------------------------------
1960
```

```
1961  // remove duplicates from sorted array
1962  var removeDuplicate = function (num) {
1963    let j = 1;
1964    for (let i = 0; i < num.length - 1; i++) {
1965      if (num[i] !== num[i + 1]) {
1966        num[j] = num[i + 1];
1967        j++;
1968      }
1969    }
1970    return j;
1971  };
1972
1973  // Example usage:
1974  let num = [0, 0, 1, 1, 1, 2, 2, 3, 3, 4];
1975  let newLength = removeDuplicates(num);
1976  console.log(`New length: ${newLength}`);
1977
1978  // ------------------------------------------------------
1979  // Merge Sorted Array
1980  let arr1 = [2, 5, 6];
1981  let arr2 = [1, 3, 4];
1982  let merged = mergeSortedArrays(arr1, arr2);
1983  console.log(merged); // Output: [1, 2, 3, 4, 5, 6]
1984  function mergeSortedArrays(arr1, arr2) {
1985    let merged = [];
1986    let i = (j = k = 0);
1987    while (i < arr1.length && j < arr2.length) {
1988      if (arr1[i] < arr2[j]) {
1989        merged[k++] = arr1[i++];
1990      } else {
1991        merged[k++] = arr2[j++];
1992      }
1993    }
1994  // Copy any remaining elements from either array
1995    while (i < arr1.length) {
1996      merged[k++] = arr1[i++];
1997    }
1998    while (j < arr2.length) {
1999      merged[k++] = arr2[j++];
2000    }
2001    return merged;
2002  }
2003
2004  // or
2005
2006  var merge = function (nums1, m, nums2, n) {
2007      let p1 = m - 1;
2008      let p2 = n - 1;
2009      let p = m + n - 1;
2010      while (p1 >= 0 && p2 >= 0) {
2011          if (nums1[p1] > nums2[p2]) {
2012              nums1[p] = nums1[p1];
```

```
                p1--;
            } else {
                nums1[p] = nums2[p2];
                p2--;
            }
            p--;
        }
    // Copy remaining elements of nums2, if any
        while (p2 >= 0) {
            nums1[p] = nums2[p2];
            p2--;
            p--;
        }
        return nums1;
    }
    // Example usage:
    let nums1 = [1, 2, 3, 0, 0, 0];
    let m = 3;
    let nums2 = [2, 5, 6];
    let n = 3;
    console.log(merge(nums1, m, nums2, n)); // Output: [1, 2, 2, 3, 5, 6]

    // ---------------------------------------------------------

    // Best Time to Buy and Sell Stock
    var maxProfit = function (prices) {
        let maxProfit = 0;
        let minPrice = Infinity;
        for (let price of prices) {
            if (price < minPrice) {
                minPrice = price; // Update the minimum price
            } else {
                maxProfit = Math.max(maxProfit, price - minPrice); // Calculate profit
            }
        }
        return maxProfit; // Return the maximum profit
    };
    // Example usage:
    let prices = [7, 1, 5, 3, 6, 4];
    console.log(maxProfit(prices)); // Output: 5 (Buy at 1, Sell at 6)

    // Sort the colors
    var sortColors = function (nums) {
        let low = 0, mid = 0, high = nums.length - 1;
        while (mid <= high) {
            if (nums[mid] === 0) {
                [nums[low], nums[mid]] = [nums[mid], nums[low]];
                low++;
                mid++;
            } else if (nums[mid] === 1) {
                mid++;
            } else {
```

```
2065                 [nums[mid], nums[high]] = [nums[high], nums[mid]];
2066                 high--;
2067             }
2068         }
2069         return nums; // Return the sorted array
2070     };
2071     // Example usage:
2072     let nums = [2, 0, 2, 1, 1, 0];
2073     console.log(sortColors(nums)); // Output: [0, 0, 1, 1, 2, 2]
2074
2075     // Kandane's Algorithm for Maximum Subarray
2076     var maxSubArray = function (nums) {
2077       let max = -Infinity;
2078       let sum = 0;
2079       for (let i = 0; i < nums.length; i++) {
2080         sum += nums[i];
2081         max = Math.max(max, sum);
2082         if (sum < 0) sum = 0;
2083       }
2084       return max;
2085     };
2086
2087     // or----------------------------------------------------------
2088     //
2089     var maxSubArray = function (nums) {
2090         let maxSum = nums[0];
2091         let currentSum = nums[0];
2092         for (let i = 1; i < nums.length; i++) {
2093             currentSum = Math.max(nums[i], currentSum + nums[i]); // Update current sum
2094             maxSum = Math.max(maxSum, currentSum); // Update max sum
2095         }
2096         return maxSum; // Return the maximum subarray sum
2097     };
2098     // Example usage:
2099     let nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4];
2100     console.log(maxSubArray(nums)); // Output: 6 (subarray [4, -1, 2, 1] has the largest sum
2101     // -------------------------------------------------------------------
2102     // majority element
2103
2104     var majorityElement = function (nums) {
2105       let ans = nums[0];
2106       let count = 1;
2107       for (let i = 1; i < nums.length; i++) {
2108         if (nums[i] === ans) count++;
2109         else count--;
2110         if (count === 0) {
2111           ans = nums[i];
2112           count = 1;
2113         } else if (ans == nums[i]) count++;
2114         else count--;
2115       }
2116       return ans;
```

```
};
console.log(majorityElement([3, 2, 3, 1, 1, 2, 3, 5]));

// OR------------------------
var majorityElement = function(nums) {
    let count = 0;
    let candidate;
    for (let num of nums) {
        if (count === 0) {
            candidate = num; // Set candidate when count is zero
        }
        count += (num === candidate) ? 1 : -1; // Increment or decrement count
    }
// Verify if candidate is indeed the majority element
    count = 0;
    for (let num of nums) {
        if (num === candidate) {
            count++;
        }
    }
    return count > nums.length / 2 ? candidate : null; // Return candidate if it's a maj
};
// Example usage:
let nums = [3, 2, 3];
console.log(majorityElement(nums)); // Output: 3
// ------------------------------------------------------------
// Trapping Rain Water
var trap = function (height) {
  let left = new Array(height.length);
  let right = new Array(height.length);
  let maxLeft = height[0],
    maxRight = height[height.length - 1];
  (left[0] = maxLeft), (right[right.length - 1] = maxRight);
  for (i = 1; i < height; i++) {
    maxLeft = Math.max(height[i], maxLeft);
    left[i] = maxLeft;
  }
  for (i = height.length - 2; i >= 0; i--) {
    maxRight = Math.max(height[i], maxRight);
    right[i] = maxRight;
  }
  let ans = 0;
  for (let i = 0; i < height.length; i++) {
    ans += Math.min(left[i], right[i] - height[i]);
  }
  return ans;
};
console.log(trap(1,2,5,6,6,6,8,8,8,9));


var trap = function(height) {
    let left = 0, right = height.length - 1;
```

```
        let leftMax = 0, rightMax = 0;
        let waterTrapped = 0;

        while (left < right) {
            if (height[left] < height[right]) {
                if (height[left] >= leftMax) {
                    leftMax = height[left];
                } else {
                    waterTrapped += leftMax - height[left];
                }
                left++;
            } else {
                if (height[right] >= rightMax) {
                    rightMax = height[right];
                } else {
                    waterTrapped += rightMax - height[right];
                }
                right--;
            }
        }
        return waterTrapped; // Return the total amount of trapped water
    }
    // Example usage:
    let height = [0,1,0,2,1,0,1,3,2,1,2,1];
    console.log(trap(height)); // Output: 6



    // ------------------------------------------------------------------------------
```